

Kotlin Observer Pattern

50 Different Implementations with Complete Code

Master the Observer pattern in Kotlin with 50 unique implementations: Classic OOP, Functional, Coroutines, Flow, LiveData, Event Bus, Weak References, Priority, Filtering, Async, Debouncing, Throttling, and much more. Each example shows a different approach to reactive programming.

BASIC PATTERNS (1-15)

Example 1: Classic Observer - Subject/Observer Interface

```
// Observer interface interface Observer { fun update(data: String) } // Subject interface interface Subject { fun attach(observer: Observer) fun
detach(observer: Observer) fun notify(data: String) } // Concrete Subject class NewsAgency : Subject { private val observers =
mutableListOf<Observer>() override fun attach(observer: Observer) { observers.add(observer) } override fun detach(observer: Observer) {
observers.remove(observer) } override fun notify(data: String) { observers.forEach { it.update(data) } } fun publishNews(news: String) {
println("Publishing: $news") notify(news) } } // Concrete Observers class NewsChannel(private val name: String) : Observer { override fun
update(data: String) { println("Name received: $data") } } // Usage fun main() { val agency = NewsAgency() val cnn = NewsChannel("CNN") val bbc =
NewsChannel("BBC") agency.attach(cnn) agency.attach(bbc) agency.publishNews("Breaking News!") }
```

■ Classic Gang of Four Observer pattern - foundation for event-driven systems

Example 2: Lambda-based Observer (Functional Approach)

```
class EventEmitter<T> { private val listeners = mutableListOf<T -> Unit>() fun subscribe(listener: (T) -> Unit): () -> Unit {
listeners.add(listener) // Return unsubscribe function return { listeners.remove(listener) } } fun emit(data: T) { listeners.forEach { it(data) } } }
// Usage fun main() { val buttonClicks = EventEmitter<String>() // Subscribe with lambdas val unsub1 = buttonClicks.subscribe { event ->
println("Handler 1: $event") } buttonClicks.subscribe { event -> println("Handler 2: $event clicked!") } buttonClicks.emit("Login Button")
buttonClicks.emit("Submit Button") // Unsubscribe unsub1() buttonClicks.emit("Cancel Button") // Handler 1 won't receive }
```

■ Functional style with lambda observers - cleaner, more flexible

Example 3: Property Observer with Delegates

```
import kotlin.properties.Delegates class User { var name: String by Delegates.observable("Unknown") { property, oldValue, newValue -> println("Name
changed from '$oldValue' to '$newValue'") } var age: Int by Delegates.vetoable(0) { property, oldValue, newValue -> println("Age change requested:
$oldValue -> $newValue") newValue >= 0 && newValue <= 150 // Veto invalid ages } } fun main() { val user = User() user.name = "Alice" // Triggers
observer user.name = "Bob" // Triggers again user.age = 25 // Accepted user.age = -5 // Vetoed (stays 25) user.age = 200 // Vetoed (stays 25)
println("Final age: ${user.age}") }
```

■ Built-in Kotlin delegates for property observation - very common pattern

Example 4: Flow-based Observer (Coroutines)

```
import kotlinx.coroutines.* import kotlinx.coroutines.flow.* class TemperatureSensor { private val _temperature = MutableStateFlow(20.0) val
temperature: StateFlow<Double> = _temperature fun setTemperature(temp: Double) { _temperature.value = temp } } fun main() = runBlocking { val sensor
= TemperatureSensor() // Observer 1: Simple collector launch { sensor.temperature.collect { temp -> println("Display: ${temp}°C") } } // Observer 2:
Alert on high temperature launch { sensor.temperature .filter { it > 30 } .collect { temp -> println("HIGH TEMP ALERT: ${temp}°C") } } delay(100)
sensor.setTemperature(25.0) delay(100) sensor.setTemperature(35.0) // Triggers alert delay(100) }
```

■ Modern Kotlin approach using StateFlow - reactive and lifecycle-aware

Example 5: LiveData Observer (Android)

```
// Simulated LiveData for demonstration class LiveData<T> { private val observers = mutableMapOf<Any, (T) -> Unit>() private var value: T? = null fun
observe(owner: Any, observer: (T) -> Unit) { observers[owner] = observer value?.let { observer(it) } } fun setValue(newValue: T) { value = newValue
observers.values.forEach { it(newValue) } } fun removeObserver(owner: Any) { observers.remove(owner) } } class UserViewModel { private val _userName
= LiveData<String>() val userName: LiveData<String> = _userName fun updateName(name: String) { _userName.setValue(name) } } // Usage
(Activity/Fragment) fun main() { val viewModel = UserViewModel() val activity = "MainActivity" viewModel.userName.observe(activity) { name ->
println("UI Updated: $name") } viewModel.updateName("Alice") viewModel.updateName("Bob") }
```

■ Android LiveData pattern - automatically handles lifecycle

Example 6: Multi-typed Event Observer

```
sealed class Event { data class UserLoggedIn(val userId: String) : Event() data class UserLoggedOut(val reason: String) : Event() data class
DataLoaded(val items: Int) : Event() data class Error(val message: String) : Event() } class EventBus { private val listeners = mutableListOf<Event>
-> Unit>() fun subscribe(listener: (Event) -> Unit) { listeners.add(listener) } fun publish(event: Event) { listeners.forEach { it(event) } } } fun
main() { val eventBus = EventBus() // Analytics observer eventBus.subscribe { event -> when (event) { is Event.UserLoggedIn -> println("Analytics:
User ${event.userId} logged in") is Event.Error -> println("Analytics: Error logged") else -> {} } } // UI observer eventBus.subscribe { event ->
when (event) { is Event.UserLoggedIn -> println("Welcome ${event.userId}!") is Event.UserLoggedOut -> println("Goodbye: ${event.reason}") is
Event.DataLoaded -> println("Loaded ${event.items} items") is Event.Error -> println("Error: ${event.message}") } }
eventBus.publish(Event.UserLoggedIn("user123")) eventBus.publish(Event.DataLoaded(42)) eventBus.publish(Event.Error("Network timeout")) }
```

■ Sealed classes for type-safe event handling - very robust

Example 7: Weak Reference Observer (Memory Safe)

```
import java.lang.ref.WeakReference interface MemoryObserver { fun onMemoryWarning(level: Int) } class MemoryManager { private val observers =
mutableListOf<WeakReference<MemoryObserver>>() fun addObserver(observer: MemoryObserver) { observers.add(WeakReference(observer)) } fun
notifyMemoryWarning(level: Int) { // Clean up dead references observers.removeAll { it.get() == null } // Notify alive observers observers.forEach {
ref -> ref.get()?.onMemoryWarning(level) } } } class ImageCache : MemoryObserver { override fun onMemoryWarning(level: Int) { println("ImageCache:
Clearing cache (level $level)") // Clear cache... } } fun main() { val memoryManager = MemoryManager() var cache: ImageCache? = ImageCache()
memoryManager.addObserver(cache!!) memoryManager.notifyMemoryWarning(1) // Cache notified cache = null // Allow garbage collection System.gc()
memoryManager.notifyMemoryWarning(2) // Cache not notified (GC'd) }
```

■ Prevents memory leaks by allowing observers to be garbage collected

Example 8: Priority-based Observer

```
data class PriorityObserver<T> ( val priority: Int, val observer: (T) -> Unit ) class PriorityEventEmitter<T> { private val observers =
mutableListOf<PriorityObserver<T>>() fun subscribe(priority: Int, observer: (T) -> Unit) { observers.add(PriorityObserver(priority, observer))
observers.sortByDescending { it.priority } } fun emit(data: T) { observers.forEach { it.observer(data) } } } fun main() { val emitter =
PriorityEventEmitter<String>() emitter.subscribe(priority = 1) { msg -> println("Low priority: $msg") } emitter.subscribe(priority = 10) { msg ->
println("High priority: $msg") } emitter.subscribe(priority = 5) { msg -> println("Medium priority: $msg") } emitter.emit("Important Message") //
Output order: High, Medium, Low }
```

■ Controls notification order - critical handlers execute first

Example 9: Filtered Observer (Conditional Notifications)

```
class FilteredEventEmitter<T> { private data class FilteredListener<T> ( val filter: (T) -> Boolean, val listener: (T) -> Unit ) private val listeners
= mutableListOf<FilteredListener<T>>() fun subscribe(filter: (T) -> Boolean = { true }, listener: (T) -> Unit) {
listeners.add(FilteredListener(filter, listener)) } fun emit(data: T) { listeners.forEach { fl -> if (fl.filter(data)) { fl.listener(data) } } }
data class LogMessage(val level: String, val message: String) fun main() { val logger = FilteredEventEmitter<LogMessage>() // Only errors
logger.subscribe(filter = { it.level == "ERROR" }) { log -> println("■ Error Logger: ${log.message}") } // Only warnings and errors
logger.subscribe(filter = { it.level in listOf("WARN", "ERROR" ) }) { log -> println("■ Alert System: ${log.message}") } // All messages
logger.subscribe { log -> println("■ Full Log: [${log.level}] ${log.message}") } logger.emit(LogMessage("INFO", "App started"))
logger.emit(LogMessage("WARN", "Low memory")) logger.emit(LogMessage("ERROR", "Connection failed")) }
```

■ Observers only receive relevant events - reduces unnecessary processing

Example 10: Async Observer with Coroutines

```
import kotlinx.coroutines.* class AsyncEventEmitter<T> { private val listeners = mutableListOf<suspend (T) -> Unit>() fun subscribe(listener: suspend
(T) -> Unit) { listeners.add(listener) } suspend fun emit(data: T) = coroutineScope { listeners.map { listener -> launch { listener(data) }
}.joinAll() } } suspend fun heavyProcessing(data: String) { delay(1000) // Simulate heavy work println("Processed: $data") } fun main() = runBlocking {
val emitter = AsyncEventEmitter<String>() emitter.subscribe { data -> println("Observer 1 started: $data") delay(500) println("Observer 1 done:
$data") } emitter.subscribe { data -> heavyProcessing(data) } emitter.subscribe { data -> println("Observer 3: $data") } println("Emitting event...")
emitter.emit("Test Data") println("All observers notified") }
```

■ Non-blocking async observers - all execute concurrently

Example 11: Debounced Observer (Rate Limiting)

```
import kotlinx.coroutines.* class DebouncedObserver<T> ( private val waitMs: Long, private val coroutineScope: CoroutineScope, private val action: (T)
-> Unit ) { private var debounceJob: Job? = null fun emit(data: T) { debounceJob?.cancel() debounceJob = coroutineScope.launch { delay(waitMs)
action(data) } } } fun main() = runBlocking { val searchObserver = DebouncedObserver<String>( waitMs = 500, coroutineScope = this ) { query ->
println("Searching for: $query") } // Rapid fire - only last one triggers searchObserver.emit("a") delay(100) searchObserver.emit("ab") delay(100)
searchObserver.emit("abc") delay(100) searchObserver.emit("abcd") // Wait for debounce delay(1000) }
```

■ Prevents excessive notifications - common in search/auto-save

Example 12: State Change Observer with Diff

```
data class UserState( val name: String, val email: String, val age: Int ) class StateObserver<T> { private var currentState: T? = null private val
listeners = mutableListOf<(old: T?, new: T) -> Unit>() fun subscribe(listener: (old: T?, new: T) -> Unit) { listeners.add(listener) } fun
setState(newState: T) { val oldState = currentState currentState = newState listeners.forEach { it(oldState, newState) } } } fun main() { val
stateObserver = StateObserver<UserState>() stateObserver.subscribe { old, new -> if (old == null) { println("Initial state: $new") } else { if
(old.name != new.name) println("Name changed: ${old.name} -> ${new.name}") if (old.email != new.email) println("Email changed: ${old.email} ->
${new.email}") if (old.age != new.age) println("Age changed: ${old.age} -> ${new.age}") } } } stateObserver.setState(UserState("Alice", "alice@ex.com",
25)) stateObserver.setState(UserState("Alice", "newemail@ex.com", 25)) stateObserver.setState(UserState("Bob", "newemail@ex.com", 26)) }
```

■ Tracks what changed - efficient UI updates, undo/redo systems

Example 13: One-time Observer (Auto-unsubscribe)

```
class EventEmitter<T> { private val listeners = mutableListOf<Listener<T>>() private data class Listener<T> ( val callback: (T) -> Unit, val once:
Boolean = false ) fun on(callback: (T) -> Unit) { listeners.add(Listener(callback, once = false)) } fun once(callback: (T) -> Unit) {
listeners.add(Listener(callback, once = true)) } fun emit(data: T) { val toRemove = mutableListOf<Listener<T>>() listeners.forEach { listener ->
listener.callback(data) if (listener.once) { toRemove.add(listener) } } listeners.removeAll(toRemove) } } fun main() { val emitter =
EventEmitter<String>() // Regular listener emitter.on { println("Regular: $it") } // One-time listener emitter.once { println("Once: $it") }
emitter.emit("Event 1") emitter.emit("Event 2") emitter.emit("Event 3") // "Once" only prints for Event 1 }
```

■ Useful for initialization events or one-off responses

Example 14: Throttled Observer (Max Rate)

```
import kotlinx.coroutines.* class ThrottledObserver<T> ( private val intervalMs: Long, private val scope: CoroutineScope, private val action: (T) ->
Unit ) { private var lastEmitTime = 0L private var throttleJob: Job? = null fun emit(data: T) { val currentTime = System.currentTimeMillis() val
timeSinceLastEmit = currentTime - lastEmitTime if (timeSinceLastEmit >= intervalMs) { lastEmitTime = currentTime action(data) } else { // Schedule
for later throttleJob?.cancel() throttleJob = scope.launch { delay(intervalMs - timeSinceLastEmit) lastEmitTime = System.currentTimeMillis()
action(data) } } } } fun main() = runBlocking { val clickObserver = ThrottledObserver<String>( intervalMs = 1000, scope = this ) { event ->
println("Click processed: $event at ${System.currentTimeMillis()}") } // Rapid clicks - throttled to max 1 per second repeat(5) { i ->
clickObserver.emit("Click $i") delay(300) } delay(2000) }
```

■ Prevents spam - useful for button clicks, scroll events

Example 15: Chained Observers (Observer Chain)

```
class Pipeline<T> { private val processors = mutableListOf<(T) -> T>() private val finalListeners = mutableListOf<(T) -> Unit>() fun
addProcessor(processor: (T) -> T) { processors.add(processor) } fun subscribe(listener: (T) -> Unit) { finalListeners.add(listener) } fun emit(data:
T) { var processedData = data processors.forEach { processor -> processedData = processor(processedData) } finalListeners.forEach { it(processedData)
} } } fun main() { val textPipeline = Pipeline<String>() // Add processors textPipeline.addProcessor { it.trim() } textPipeline.addProcessor {
it.lowercase() } textPipeline.addProcessor { it.replace("bad", "****") } // Add observers textPipeline.subscribe { processed -> println("Display:
$processed") } textPipeline.subscribe { processed -> println("Length: ${processed.length}") } textPipeline.emit(" Hello BAD World ") // Output:
"hello *** world" }
```

■ *Middleware pattern - data transformation before notification*